

Ломбард «Драгоценности Урала»

Инструкция. Сервер «ПБД»

Сервер промежуточной базы данных

По состоянию на
4.11.2025

Оглавление

1. Общее описание системы	4
Содержание файла помощи	4
Возможности файла помощи	5
2. Требования к системе.....	5
3. Установка и настройка	5
4. Запуск сервера	6
4.1 Режимы работы сервера	6
4.2 Обычный режим работы.....	6
4.3 Подробный режим (verbose).....	7
4.4 Режим проверки статуса	7
4.4.1 Запуск режима проверки	7
4.4.2 Проверка конфигурационного файла.....	7
4.4.3 Диагностика подключения к базе данных MSSQL	8
4.4.4 Верификация системы безопасности	8
4.4.5 Диагностика подсистемы логирования.....	8
4.4.6 Проверка облачного хранилища	9
4.4.7 Проверка сетевых настроек	9
4.4.8 Формат вывода результатов	9
4.4.9 Интерпретация результатов	9
4.5 Детальный процесс запуска сервера	9
4.5.1 Этапы инициализации сервера	10
4.5.2 Критические зависимости между этапами	11
4.5.3 Обработка ошибок при запуске	12
4.6 Сообщения при успешном запуске	12
4.6.1 Структура сообщений о запуске	12
4.6.2 Последовательность сообщений при запуске	12
4.6.3 Детальное описание сообщений о статусе	13
4.6.4 Сообщения в подробном режиме (verbose)	14
4.6.5 Обработка ошибок запуска	14
4.7 Обработка ошибок при запуске	14
4.7.1 Типы ошибок и их диагностика	15
4.7.2 Обязательные параметры конфигурации	16

4.7.3	Последовательность проверок при запуске	16
4.7.4	Рекомендации по устранению ошибок	16
4.8	Особенности запуска в различных средах	17
4.8.1	Конфигурации для различных сред	17
4.8.2	Дополнительные параметры конфигурации для сред	17
4.8.3	Рекомендуемые настройки безопасности по средам.....	17
4.8.4	Настройки логирования для различных сред	18
4.8.5	Особенности сетевых настроек	18
4.8.6	Рекомендации по производительности	18
5.	Конфигурация безопасности	19
5.1	Базовые настройки безопасности	19
5.2	Режимы отключения безопасности.....	19
5.3	Безопасность эндпоинтов.....	20
5.4	Защита от перебора паролей	20
5.5	Алгоритм работы цифровых подписей.....	20
5.6	Серверные подписи ответов	21
5.7	Рекомендации по настройке безопасности	21
6.	Работа с базой данных	21
6.1	Конфигурирование подключения к базе данных	22
6.2	Дополнительные настройки базы данных	22
6.3	Управление пулом соединений	22
6.4	Мониторинг состояния базы данных	23
6.5	Пример конфигурации подключения к БД.....	23
6.6	Проверка подключения к базе данных.....	24
6.7	Основные хранимые процедуры	24
6.8	Обработка ошибок базы данных	24
7.	Логирование и мониторинг	25
7.1	Логирование в базу данных	25
7.2	Логирование в файл.....	26
7.3	Консольное логирование	26
7.4	Уровни логирования и фильтрация	27
7.5	Мониторинг состояния системы.....	27
8.	API эндпоинты	28

8.1 Эндпоинты мониторинга и диагностики	28
GET /health	28
GET /health/security	28
GET /health/database	29
GET /health/logging	29
GET /health/network	29
GET /health/cloud	30
8.2 Эндпоинты управления пользователями	30
POST /user/by-phone	34
POST /user/update	34
POST /user/login	35
POST /user/mailling	35
8.3 Эндпоинты работы с залоговыми операциями	36
POST /ticket/list	36
POST /payment/set	37
8.4 Эндпоинты работы с документами	37
POST /document/list	37
POST /document/load	38
POST /document/signed	38
8.5 Вспомогательные эндпоинты	39
GET /help	39
8.6 Общие требования ко всем запросам	39
9. Облачное хранилище	39
9.1 Назначение и возможности	39
9.2 Конфигурационные параметры	40
9.3 Пример конфигурации	41
9.4 Процесс работы с облачным хранилищем	42
9.5 Мониторинг состояния облачного хранилища	42
9.6 Рекомендации по настройке	42
9.7 Обработка ошибок	43
10. Диагностика и устранение неисправностей	43
10.1 Проверка состояния сервера	43
10.2 Типичные проблемы и решения	43

1. Общее описание системы

Сервер безопасного обмена данными — это веб-сервер на Python, предназначенный для защищенного обмена информацией между системами. Основные функции включают аутентификацию по Bearer token и RSA цифровым подписям, многоуровневое логирование и health-check мониторинг.

Архитектура системы состоит из следующих компонентов:

- Конфигурация - класс Config для хранения параметров сервера
- Безопасность - аутентификация, RSA шифрование, валидация подписей
- База данных - работа с MSSQL через pyodbc
- Логирование - многоуровневое логирование (БД, файл, консоль)
- Middleware - глобальная обработка запросов
- API endpoints - обработчики HTTP запросов

Сервер оснащен инструкцией и удобными скриптами диагностики, так же имеется функционал администрирования конфигурационного файла, обратится к данному функционалу можно через браузер к адресу [http://\[адрес сервера\]:\[порт сервера\]/help](http://[адрес сервера]:[порт сервера]/help)

Содержание файла помощи

Данный HTML-файл представляет собой полную интерактивную документацию для Secure Data Exchange Server - защищенного сервера обмена данными. Документация включает:

- Основные разделы:
 - Обзор системы - описание назначения и архитектурных особенностей
 - Состояние системы - интерактивный мониторинг всех компонентов сервера
 - Установка и запуск - системные требования и инструкции по развертыванию
 - Конфигурация - детальное описание настроек сервера
 - Система безопасности - принципы работы системы защиты
- Техническая документация:
 - Служебные эндпоинты - health-чекеры и сервисные endpoints
 - API эндпоинты - полное описание всех рабочих endpoints, Полная API документация (Детальное описание каждого эндпоинта, Параметры запросов и ответов, Уровни безопасности, Бизнес-логика и SQL-запросы)
- Конфигурационные параметры - детальные настройки безопасности, БД, логирования, CORS, облачного хранилища

Возможности файла помощи

- Интерактивные функции:
 - Реал-тайм мониторинг системы
 - Автоматическое обновление статуса каждые 30 секунд
 - Детальная диагностика всех компонентов (сервер, БД, безопасность, логирование, сеть, облако)
 - Визуальные индикаторы состояния (онлайн/офлайн/предупреждение)
- Вкладки состояния системы: Общее состояние, Безопасность, База данных, Логирование, Сеть, Облачное хранилище
- Редактор Конфигурации - JSON-редактор с валидацией и форматированием, Визуальные формы для всех настроек, Скачивание готовой конфигурации.

ВНИМАНИЕ! Редактор Конфигурации не вносит изменения в настройки сервера напрямую. Его задача — формировать JSON-структуру с параметрами, которую необходимо вручную скопировать в ваш конфигурационный файл.

Порядок применения новых настроек:

- Сгенерируйте и сохраните JSON в Редакторе.
- Вставьте полученную структуру в ваш конфигурационный файл.
- Перезапустите сервер для вступления изменений в силу.

Примечание: если в конфигурации задан интервал автоматической перезагрузки, новые настройки применятся после его истечения, и перезапуск сервера вручную не потребуется.

Этот файл помощи представляет собой полноценное веб-приложение для управления и документирования сервера, сочетающее в себе функции документации, мониторинга и конфигурации.

2. Требования к системе

Для работы сервера необходимы следующие компоненты:

Python	Версия 3.7 или выше
Зависимости	aiohttp, pyodbc, cryptography, requests
База данных	MSSQL Server с ODBC Driver 18
Операционная система	Windows/Linux/macOS
Память	Не менее 512 МБ ОЗУ
Сеть	Открытый порт для входящих подключений

3. Установка и настройка

Перед запуском сервера необходимо выполнить следующие шаги:

1. Установите Python 3.7 или выше
2. Установите необходимые зависимости:

- `pip install aiohttp pyodbc cryptography requests`
- 3. Настройте подключение к MSSQL Server
- 4. Подготовьте RSA ключи для безопасности:
 - `server_private_key_path` - приватный ключ сервера
 - `client_public_key_path` - публичный ключ клиента
- 5. Создайте конфигурационный файл `config.json`

4. Запуск сервера

Сервер представляет собой асинхронное веб-приложение на Python, предназначенное для безопасного обмена данными между системами. Поддерживает несколько режимов работы, которые выбираются при запуске через аргументы командной строки. Сервер обеспечивает аутентификация по Bearer token и RSA цифровым подписям, многоуровневое логирование в MSSQL БД, файл и консоль, health-check мониторинг состояния сервера и БД, а также подробный verbose-режим для отладки запросов и ответов.

4.1 Режимы работы сервера

Сервер поддерживает три основных режима работы, каждый из которых предназначен для конкретных сценариев использования:

Обычный режим	<code>python3 server.py config.json</code>	Базовое логирование ошибок и информации о запуске, рекомендуется для производственной среды
Подробный режим	<code>python3 server.py config.json --verbose</code>	Полное логирование всех запросов и ответов для отладки и мониторинга
Проверка статуса	<code>python3 server.py --status</code>	Проверка состояния сервера без его полного запуска, диагностика конфигурации

4.2 Обычный режим работы

В обычном режиме сервер запускается с базовым логированием, что обеспечивает максимальную производительность в производственной среде. Данный режим рекомендуется для промышленной эксплуатации, когда подробное логирование не требуется.

Команда запуска: `python3 server.py config.json`

В этом режиме функционируют следующие системы логирования:

- Логирование в файл - записываются события согласно настройкам уровней в `config.json` (INFO, ERROR и другие)
- Логирование в базу данных - сохраняются события согласно настройкам уровней в `config.json`
- Консольный вывод - полностью отключен для минимизации нагрузки и повышения производительности

- Маскирование чувствительных данных - пароли, токены и подписи автоматически маскируются в логах
- Health-check эндпоинты - предоставляют информацию о состоянии логирования через API запросы

4.3 Подробный режим (verbose)

Подробный режим включает расширенное логирование всех операций сервера, что незаменимо при отладке, разработке и расследовании инцидентов. В этом режиме администратор получает полную видимость всех процессов, происходящих в системе.

Команда запуска: `python3 server.py config.json --verbose`

В подробном режиме отображается следующая информация:

- Полные данные о каждом входящем HTTP запросе - метод, URL, заголовки, параметры, тело запроса
- Детальная информация об исходящих HTTP ответах - статус код, заголовки, тело ответа
- Процесс аутентификации - проверка токенов и цифровых подписей, диагностика ошибок безопасности
- Время обработки каждого запроса - точное время в миллисекундах, затраченное на обработку
- Работа с базой данных - выполнение SQL запросов, вызов хранимых процедур, время выполнения
- Взаимодействие с облачным хранилищем - процесс загрузки файлов, создание публичных ссылок
- Генерация серверных подписей - процесс создания цифровых подписей для ответов клиентам

4.4 Режим проверки статуса

Режим проверки статуса предоставляет администратору возможность выполнить комплексную диагностику системы без ее полного запуска. Данный функционал критически важен для предварительной проверки корректности конфигурации и доступности всех зависимых сервисов перед развертыванием в рабочей среде.

4.4.1 Запуск режима проверки

Для активации режима диагностики выполните команду:

`python3 server.py --status` Система выполнит последовательную проверку всех компонентов и выведет детализированный отчет о состоянии.

4.4.2 Проверка конфигурационного файла

Система осуществляет глубокий анализ конфигурационного файла, проверяя не только синтаксическую корректность JSON, но и семантическую целостность параметров:

- Валидация структуры конфигурации согласно ожидаемой схеме
- Проверка обязательных параметров в каждом разделе конфигурации
- Верификация типов данных и допустимых значений параметров
- Проверка корректности путей к файлам и директориям
- Валидация форматов URL, портов, временных интервалов

4.4.3 Диагностика подключения к базе данных MSSQL

Выполняется многоуровневая проверка подключения к Microsoft SQL Server:

Сетевой уровень	Доступность сервера по указанному host:port
Аутентификация	Проверка корректности username/password
Драйвер ODBC	Наличие и работоспособность указанного драйвера
База данных	Доступность указанной базы данных
Таймауты	Проверка параметров времени ожидания подключения
Тестовый запрос	Выполнение SELECT 1 для подтверждения работоспособности

4.4.4 Верификация системы безопасности

Комплексная проверка криптографических компонентов системы:

1. Приватный ключ сервера (server_private_key_path)
 - Проверка существования файла
 - Валидация формата PEM
 - Проверка возможности загрузки ключа
 - Тестирование возможности расшифровки данных
2. Публичный ключ клиента (client_public_key_path)
 - Проверка существования файла
 - Валидация формата PEM
 - Проверка возможности загрузки ключа
 - Тестирование возможности шифрования данных
3. Проверка токенов безопасности
 - Валидация списка allowed_tokens
 - Проверка наличия default_token_server
 - Верификация формата токенов

4.4.5 Диагностика подсистемы логирования

Проверка всех компонентов системы логирования:

Файловое логирование	Проверка доступности log_file_path, прав на запись, корректности уровней логирования
Логирование в БД	Проверка доступности таблицы Logs, корректности структуры, уровней логирования
Консольное логирование	Проверка настроек verbose режима
Маскирование данных	Проверка работы механизма маскирования конфиденциальных данных

4.4.6 Проверка облачного хранилища

При активированной облачной интеграции выполняется полная диагностика:

- Доступность облачного сервера по URL
- Корректность аутентификационных данных (username/password)
- Доступность репозитория (repo_id)
- Проверка путей загрузки (upload_path)
- Доступность временной директории (temp_dir)
- Тестирование возможности создания публичных ссылок

4.4.7 Проверка сетевых настроек

Валидация конфигурации сети и CORS:

1. Проверка корректности host и port для запуска сервера
2. Валидация настроек CORS (allowed_origins, methods, headers)
3. Проверка параметров max_age и allow_credentials
4. Тестирование доступности эндпоинтов

4.4.8 Формат вывода результатов

Результаты диагностики выводятся в структурированном виде с цветовой кодировкой:

OK	Компонент функционирует корректно, готов к работе
ERROR	Критическая ошибка, требующая немедленного вмешательства
WARNING	Некритическая проблема, требующая внимания администратора
INFO	Информационные сообщения о состоянии компонентов

4.4.9 Интерпретация результатов

По завершении диагностики система формирует итоговый отчет, содержащий:

- Сводку по количеству успешных и неуспешных проверок
- Детализированный список обнаруженных проблем
- Конкретные рекомендации по устранению каждой проблемы
- Оценку общей готовности системы к запуску
- Перечень необходимых действий для приведения системы в рабочее состояние

Режим проверки статуса позволяет администратору заблаговременно выявить и устранить потенциальные проблемы, обеспечивая стабильную и надежную работу сервера в производственной среде.

4.5 Детальный процесс запуска сервера

При запуске сервер выполняет строго определенную последовательность инициализации компонентов, обеспечивающую надежную и предсказуемую работу системы. Каждый этап

критически важен для корректного функционирования сервера и должен быть выполнен в указанном порядке.

4.5.1 Этапы инициализации сервера

Этап	Описание процесса	Критические параметры
1. Парсинг аргументов командной строки	Сервер анализирует переданные параметры запуска, включая путь к конфигурационному файлу и флаги режимов работы. Поддерживаются аргументы: config.json для обычного режима, --verbose для подробного логирования, --status для проверки статуса сервера.	Обязательный параметр: путь к config.json; Опциональные: --verbose, --status
2. Загрузка и валидация конфигурационного файла	Система загружает JSON-конфигурацию и проверяет ее корректность. Проверяются обязательные параметры безопасности, настройки БД, логирования и сетевые параметры. При отсутствии обязательных параметров сервер завершает работу с ошибкой.	Обязательные параметры: allowed_tokens, настройки БД; Проверка типов данных и значений
3. Инициализация подключения к базе данных MSSQL	Сервер устанавливает соединение с SQL Server через драйвер pyodbc используя параметры из конфигурации. Формируется строка подключения с указанием сервера, порта, базы данных, учетных данных и таймаутов.	Параметры БД: server, port, database, username, password, driver; Таймаут подключения: connection_timeout
4. Загрузка RSA ключей безопасности	Происходит асинхронная загрузка криптографических ключей: приватного ключа сервера из server_private_key_path для расшифровки клиентских подписей и публичного ключа клиента из client_public_key_path для шифрования серверных ответов.	Пути к ключам: server_private_key_path, client_public_key_path; Формат: PEM без пароля

5. Настройка систем логирования	Инициализируется многоуровневая система логирования с поддержкой записи в файл, базу данных и консоль. Настраиваются форматы сообщений, маскирование конфиденциальных данных, ротация логов по датам.	Уровни логирования: INFO, ERROR; Настройки: log_to_db, log_to_file, mask_sensitive_data
6. Проверка доступности облачного хранилища	Если облачное хранилище включено в конфигурации, сервер выполняет тестовое подключение к облачному API, проверяет аутентификационные данные и доступность репозитория для загрузки файлов.	Параметры облака: url, username, password, repo_id; Таймаут: cloud_timeout
7. Регистрация middleware и обработчиков запросов	Настраивается цепочка обработки запросов: CORS middleware для кросс-доменных запросов, auth middleware для аутентификации, регистрируются API endpoints для обработки бизнес-логики.	Middleware: CORS, аутентификация; Endpoints: health, user, tickets, payment, login, document
8. Запуск HTTP сервера на указанном хосте и порту	Запускается aiohttp веб-сервер который начинает прослушивать входящие HTTP соединения на указанном сетевом интерфейсе и порту. Сервер переходит в режим ожидания запросов.	Сетевые параметры: host, port; Протокол: HTTP
9. Инициализация периодических задач	Если в конфигурации включена автоматическая перезагрузка, настраивается периодическая задача которая будет обновлять конфигурацию сервера без необходимости перезапуска.	Интервал перезагрузки: config_reload_interval_minutes; 0 - отключено

4.5.2 Критические зависимости между этапами

Процесс запуска имеет строгие зависимости между этапами. Нарушение последовательности может привести к некорректной работе сервера или ошибкам инициализации:

- Этапы 1-2 являются обязательными и выполняются последовательно - без корректной конфигурации сервер не может продолжать инициализацию
- Этап 3 (инициализация БД) может быть пропущен если установлен allow_start_without_db, но это ограничит функциональность связанную с данными

- Этапы 4-5 выполняются параллельно так как не зависят друг от друга, но оба требуют завершения этапа 2
- Этап 6 выполняется только если `cloud_enabled=true` в конфигурации, при ошибке может быть пропущен если `allow_start_without_cloud=true`
- Этапы 7-8 выполняются последовательно после успешного завершения всех предыдущих этапов
- Этап 9 является фоновой задачей и не блокирует основной запуск сервера

4.5.3 Обработка ошибок при запуске

Сервер реализует комплексную систему обработки ошибок на каждом этапе запуска. В зависимости от критичности ошибки сервер может:

1. Завершиться с кодом ошибки - при невозможности загрузить конфигурацию или обязательные компоненты
2. Продолжить работу с ограниченной функциональностью - при недоступности необязательных компонентов (БД, облачное хранилище)
3. Вывести предупреждения и продолжить работу - при незначительных проблемах конфигурации

Все ошибки и предупреждения процесса запуска подробно логируются в соответствии с настройками логирования, что позволяет администратору оперативно диагностировать проблемы инициализации системы.

4.6 Сообщения при успешном запуске

При корректном запуске сервера в консоли отображается последовательность информационных сообщений, которые подтверждают успешную инициализацию каждого компонента системы. Каждое сообщение имеет стандартизированный формат с указанием статуса операции, типа компонента и дополнительной информации о результате выполнения.

4.6.1 Структура сообщений о запуске

Все сообщения о статусе запуска следуют единому формату визуального представления:

Префикс статуса	[OK], [ERROR], [INFO] в цветовом оформлении	Визуальная идентификация типа сообщения
Описание операции	Краткое текстовое описание выполняемой операции	Понятное обозначение проверяемого компонента
Детали выполнения	Дополнительная информация о результате операции	Конкретные данные о загруженных ресурсах

4.6.2 Последовательность сообщений при запуске

Система выводит сообщения в строгой последовательности инициализации компонентов:

1. Загрузка и валидация конфигурации
2. Подключение к базе данных MSSQL
3. Инициализация системы безопасности (RSA ключи)

4. Настройка системы логирования
5. Проверка облачного хранилища (при включении)
6. Запуск HTTP сервера и открытие портов

4.6.3 Детальное описание сообщений о статусе

Конфигурация	[OK] Конфигурация загружена из config.json	Указывает на успешную загрузку и парсинг файла конфигурации, включая все параметры безопасности, базы данных, логирования и сетевых настроек. Сообщение подтверждает отсутствие синтаксических ошибок в JSON файле и корректность всех обязательных параметров.
База данных	[OK] Подключение к MSSQL серверу установлено	Подтверждает успешное соединение с SQL Server используя указанные в конфигурации параметры: сервер, порт, имя базы данных, драйвер подключения. Дополнительно выполняется проверка доступности базы данных через тестовый запрос SELECT 1.
Версия сервера БД	[OK] Версия сервера: Microsoft SQL Server ...	Отображает информацию о версии SQL Server, что позволяет администратору убедиться в подключении к правильному экземпляру базы данных и совместимости систем.
RSA ключ сервера	[OK] Приватный ключ сервера загружен	Информирует о успешной загрузке приватного RSA ключа из указанного в конфигурации PEM файла. Ключ используется для расшифровки клиентских цифровых подписей и должен соответствовать формату PKCS#8 без пароля.
RSA ключ клиента	[OK] Публичный ключ клиента загружен	Подтверждает загрузку публичного RSA ключа клиента для проверки цифровых подписей входящих запросов. Ключ должен быть в PEM формате и соответствовать приватному ключу используемому клиентами для подписи.
Файловое логирование	[OK] Файловое логирование инициализировано	Сообщает о готовности системы логирования в файл с указанием полного пути к файлу лога. Файл автоматически создается с датой в имени (формат ГГГГ-ММ-ДД) и настраивается на запись сообщений указанных в конфигурации уровней.
Облачное хранилище	[OK] Облачное хранилище доступно, токен получен	Информирует о успешном подключении к облачному сервису через API аутентификации. Сообщение появляется только при включенной опции cloud.enabled в конфигурации и успешном получении токена доступа от облачного сервиса.

HTTP сервер	[OK] Сервер запущен на http://localhost:8000	Финальное сообщение о успешном запуске веб-сервера aiohttp с указанием точного URL для подключения. Сервер начинает прослушивать указанный порт и готов принимать входящие HTTP запросы согласно настроенным эндпоинтам.
-------------	---	--

4.6.4 Сообщения в подробном режиме (verbose)

При запуске с параметром `--verbose` система выводит дополнительную диагностическую информацию:

- Подробные параметры конфигурации с маскированием чувствительных данных
- Статистику загруженных RSA ключей (размер, алгоритм)
- Информацию о настройках CORS и доступных эндпоинтах
- Детали подключения к базе данных (таймауты, драйвер)
- Список включенных уровней логирования для каждого канала

4.6.5 Обработка ошибок запуска

В случае неудачи инициализации любого компонента система выводит сообщение об ошибке с точным указанием проблемы:

Ошибка конфигурации	[ERROR] Ошибка загрузки конфигурации: Файл не найден	Указывает на отсутствие файла config.json или ошибки формата JSON
Ошибка БД	[ERROR] Ошибка подключения к MSSQL: Неверный логин/пароль	Сообщает о проблемах аутентификации при подключении к SQL Server
Ошибка ключей	[ERROR] Ошибка загрузки приватного ключа: Неверный формат	Указывает на проблемы с форматом или доступностью RSA ключей

После успешного вывода всех сообщений о запуске сервер переходит в режим ожидания входящих HTTP запросов и начинает обработку подключений согласно настроенной бизнес-логике и правилам безопасности.

4.7 Обработка ошибок при запуске

При возникновении ошибок во время запуска сервер прекращает работу с детальным описанием проблемы, что позволяет администратору быстро идентифицировать и устранить причину. Система использует многоуровневую проверку компонентов, где каждая последующая инициализация зависит от успешности предыдущей, обеспечивая четкую диагностику проблем на каждом этапе запуска.

4.7.1 Типы ошибок и их диагностика

Отсутствие конфигурационного файла	Файл config.json не найден в указанном пути или отсутствуют права на чтение	Система проверяет существование файла, доступность для чтения и корректность пути. При отсутствии файла выводится полный путь поиска и рекомендации по созданию конфигурации.
Ошибки подключения к базе данных	Недоступность SQL Server, неверные учетные данные, отсутствие базы данных, сетевые проблемы	Проверяется доступность сервера по указанному адресу и порту, валидность логина и пароля, существование указанной базы данных. При ошибке выводится детальная информация о параметрах подключения.
Невалидные RSA ключи или отсутствие ключевых файлов	Поврежденные ключи, неправильный формат, отсутствие файлов в указанных путях	Система проверяет существование файлов ключей, их формат PEM, корректность структуры RSA ключей и соответствие алгоритмам шифрования. При ошибке указывается точный путь к проблемному файлу.
Занятый порт или проблемы с сетевыми настройками	Указанный порт уже используется другим процессом, отсутствие прав на прослушивание порта	Выполняется попытка биндинга к указанному порту на заданном интерфейсе. При занятости порта выводится информация о конфликтующем процессе и рекомендации по освобождению порта.
Ошибки синтаксиса в конфигурационном файле	Невалидный JSON, отсутствие обязательных параметров, неверные типы значений	Производится полный парсинг JSON файла с проверкой всех обязательных секций: security, database, logging, cors. Указывается точная строка и параметр с ошибкой.
Проблемы с логированием	Недоступность пути для файлов логов, отсутствие прав на запись, ошибки инициализации логгера	Проверяется возможность создания директорий для логов, права на запись в указанную директорию, корректность формата файла лога. При ошибке выводится полный путь и требуемые права доступа.
Ошибки облачного хранилища	Недоступность облачного сервиса, неверные учетные данные, проблемы с аутентификацией	Выполняется тестовое подключение к облачному API, проверка валидности учетных данных, доступности репозитория. При ошибке указывается код ответа API и детали проблемы аутентификации.

4.7.2 Обязательные параметры конфигурации

Система проверяет наличие и корректность следующих обязательных параметров в конфигурационном файле:

- Секция security с параметрами server_private_key_path и client_public_key_path
- Секция database с параметрами server, port, database, username, password
- Список allowed_tokens в секции security не может быть пустым
- Параметры host и port в основной секции конфигурации
- Секция logging с параметрами log_to_db и log_to_file

4.7.3 Последовательность проверок при запуске

1. Проверка существования и читаемости файла config.json
2. Валидация JSON синтаксиса и структуры конфигурации
3. Проверка обязательных параметров и их типов данных
4. Подключение к базе данных MSSQL с проверкой доступности
5. Загрузка и валидация RSA ключей безопасности
6. Инициализация системы логирования с проверкой прав доступа
7. Проверка доступности облачного хранилища (при включенной опции)
8. Запуск HTTP сервера с проверкой доступности порта

4.7.4 Рекомендации по устранению ошибок

Ошибки базы данных	Проверить доступность SQL Server, правильность учетных данных, наличие базы данных, настройки брандмауэра	Использовать параметр allow_start_without_db для запуска без БД
Проблемы с ключами	Убедиться в правильности формата PEM файлов, проверить пути к ключам, верифицировать соответствие ключей	Использовать disable_certificates для отключения проверки ключей
Ошибки порта	Проверить занятость порта командой netstat, изменить порт в конфигурации, проверить права доступа	Использовать порты выше 1024 для избежания проблем с правами
Проблемы логирования	Проверить права на запись в директорию логов, доступное место на диске, корректность пути	Указать относительные пути для автоматического создания директорий

Каждая ошибка сопровождается подробным сообщением на русском языке с указанием точной причины проблемы, путями к проблемным файлам, параметрами конфигурации и конкретными рекомендациями по устранению. Система обеспечивает максимально информативную диагностику для быстрого решения проблем запуска.

4.8 Особенности запуска в различных средах

Сервер поддерживает гибкую настройку параметров запуска для различных сред эксплуатации через конфигурационный файл. Каждая среда требует специфических настроек безопасности, логирования и обработки ошибок для оптимальной работы и соответствия требованиям эксплуатации.

4.8.1 Конфигурации для различных сред

Разработка	debug: true, verbose: true, allow_start_without_db: true	Максимальная отладочная информация, допуск к запуску без БД, отладочный режим веб-сервера, подробное логирование всех запросов и ответов, минимальные требования к безопасности для удобства тестирования
Тестирование	debug: false, verbose: true, allow_start_without_db: false	Подробное логирование с строгими требованиями к доступности БД, полная проверка безопасности, детальный мониторинг производительности, имитация промышленных условий с максимальной диагностикой
Промышленная эксплуатация	debug: false, verbose: false, allow_start_without_db: false	Минимальное логирование, строгая проверка всех зависимостей, максимальный уровень безопасности, оптимизация производительности, только критическое логирование ошибок

4.8.2 Дополнительные параметры конфигурации для сред

Помимо основных параметров, для каждой среды рекомендуется настраивать дополнительные опции:

- Настройки безопасности - уровень проверки сертификатов, требования к подписям, настройки CORS
- Параметры базы данных - таймауты подключения, размеры пулов соединений, настройки повторных попыток
- Конфигурация логирования - уровни детализации, ротация логов, маскирование чувствительных данных
- Настройки производительности - лимиты запросов, размеры буферов, таймауты обработки

4.8.3 Рекомендуемые настройки безопасности по средам

Разработка	disable_certificates: false, disable_token_auth: false, disable_signature: false	Полная проверка безопасности для тестирования клиентских приложений, но с возможностью быстрого отключения при необходимости
------------	--	--

Тестирование	disable_certificates: false, disable_token_auth: false, disable_signature: false	Строгая проверка всех механизмов безопасности, идентичная промышленной среде, для валидации корректности работы
Промышленная эксплуатация	disable_certificates: false, disable_token_auth: false, disable_signature: false	Максимальный уровень безопасности с обязательной проверкой сертификатов, токенов и цифровых подписей для всех запросов

4.8.4 Настройки логирования для различных сред

Разработка	log_to_db: ['DEBUG', 'INFO', 'ERROR'], log_to_file: ['DEBUG', 'INFO', 'ERROR']	Полное логирование всех событий в базу данных и файл для детального анализа работы системы во время разработки
Тестирование	log_to_db: ['INFO', 'ERROR'], log_to_file: ['DEBUG', 'INFO', 'ERROR']	Баланс между детализацией и производительностью, с сохранением отладочной информации в файлы для послетестового анализа
Промышленная эксплуатация	log_to_db: ['ERROR'], log_to_file: ['INFO', 'ERROR']	Минимальное логирование в базу данных для сохранения производительности, с сохранением информационных сообщений и ошибок в файлы для мониторинга

4.8.5 Особенности сетевых настроек

Для различных сред рекомендуются различные настройки CORS и сетевой безопасности:

- Разработка - разрешенные источники ['*'], для удобства тестирования с различных доменов
- Тестирование - конкретные домены тестовых сред, для проверки корректности работы CORS
- Промышленная эксплуатация - строго определенные домены клиентских приложений

4.8.6 Рекомендации по производительности

Разработка	signature_ttl: 3600, db_connection_timeout: 30	Увеличенные таймауты для удобства отладки, длительное время жизни подписей
Тестирование	signature_ttl: 300, db_connection_timeout: 15	Баланс между производительностью и удобством тестирования, стандартные значения
Промышленная эксплуатация	signature_ttl: 300, db_connection_timeout: 10	Оптимальные настройки для максимальной производительности и безопасности

Каждая среда эксплуатации требует тщательной настройки параметров сервера для обеспечения оптимального баланса между производительностью, безопасностью и удобством отладки. Рекомендуется использовать отдельные конфигурационные файлы для каждой среды и автоматизировать процесс их развертывания.

5. Конфигурация безопасности

Система безопасности сервера обеспечивает многоуровневую защиту данных и контролируемый доступ к API. Конфигурация безопасности настраивается в разделе "security" JSON-файла конфигурации и включает аутентификацию по токенам, цифровые подписи, управление доступом к эндпоинтам и защиту от несанкционированного доступа.

5.1 Базовые настройки безопасности

Основные параметры безопасности определяют глобальное поведение системы защиты и включают настройки ключей, токенов и временных ограничений.

server_private_key_path	Путь к файлу приватного RSA ключа сервера в формате PEM. Используется для расшифровки клиентских подписей и создания серверных подписей ответов.
client_public_key_path	Путь к файлу публичного RSA ключа клиента в формате PEM. Используется для проверки клиентских подписей и шифрования серверных подписей.
signature_ttl	Время жизни цифровой подписи в секундах (по умолчанию 300). Определяет период, в течение которого подпись считается действительной после создания.
default_token_server	Токен по умолчанию, который используется сервером для ответов, если токен не передан в запросе. Если не указан, используется первый токен из allowed_tokens.
allowed_tokens	Список разрешенных Bearer токенов для аутентификации. Минимум один токен должен быть указан, иначе сервер не запустится.

5.2 Режимы отключения безопасности

Для целей разработки и тестирования предусмотрены режимы отключения отдельных компонентов безопасности. В production среде использование этих настроек не рекомендуется.

disable_certificates	Полное отключение работы с RSA ключами. При значении true сервер не будет загружать и использовать криптографические ключи.
disable_token_auth	Отключение проверки Bearer токенов. Запросы будут обрабатываться без проверки заголовка Token.
disable_signature	Отключение проверки цифровых подписей. Сервер не будет проверять заголовок Signature в запросах и не будет добавлять подписи к ответам.

5.3 Безопасность эндпоинтов

Система позволяет настраивать индивидуальные уровни безопасности для каждого API эндпоинта. Это обеспечивает гибкость в управлении доступом к различным функциям системы.

endpoint_security	Словарь, где ключ - путь эндпоинта (например, "health" или "user/by-phone"), а значение - уровень безопасности. Поддерживаемые уровни: "public", "token", "signature", "disabled".
-------------------	--

Уровни безопасности эндпоинтов:

- **public** - открытый доступ без аутентификации. Токен проверяется опционально, если передан.
- **token** - требуется только валидный Bearer токен. Проверка цифровой подписи не выполняется.
- **signature** - требуется и токен, и валидная цифровая подпись (режим по умолчанию для всех эндпоинтов).
- **disabled** - эндпоинт полностью отключен, возвращает ошибку 404.

5.4 Защита от перебора паролей

Система включает механизм защиты от brute-force атак на аутентификацию, который отслеживает неудачные попытки входа и временно блокирует учетные записи.

enabled	Включение/отключение системы защиты от перебора паролей.
max_failed_attempts	Максимальное количество неудачных попыток входа до блокировки (по умолчанию 5).
check_period_minutes	Период проверки в минутах, за который учитываются неудачные попытки (по умолчанию 60).
allow_successful_login_during_lockout	Разрешить успешный вход при блокировке, если пароль верный. При значении false блокировка предотвращает любые попытки входа.

5.5 Алгоритм работы цифровых подписей

Система цифровых подписей обеспечивает защиту от повторного использования запросов и подтверждение подлинности сторон. Подпись создается по алгоритму RSA с использованием временных меток.

Формат данных для подписи: "токен.время_истечения", где время_истечения - Unix timestamp в секундах. Клиент подписывает эти данные своим приватным ключом, а сервер проверяет подпись публичным ключом клиента.

Процесс проверки подписи:

1. Сервер получает заголовки Token и Signature из запроса
2. Декодирует Base64 подпись из заголовка Signature
3. Проверяет подпись с использованием публичного ключа клиента
4. Извлекает из подписи токен и время истечения
5. Сравнивает токен из подписи с токеном из заголовка Token
6. Проверяет, что время истечения не наступило и не превышает 24 часа вперед
7. Если все проверки пройдены - запрос считается аутентифицированным

5.6 Серверные подписи ответов

Сервер добавляет цифровые подписи ко всем исходящим ответам, что позволяет клиенту убедиться в подлинности ответа. Для создания подписи сервер использует публичный ключ клиента.

Заголовки ответа с безопасностью:

- **Token:** Bearer [токен] - токен для следующего запроса
- **Signature:** [base64_подпись] - цифровая подпись ответа

5.7 Рекомендации по настройке безопасности

Для обеспечения максимальной защиты системы рекомендуется следовать следующим принципам:

- В production среде всегда устанавливать disable_certificates, disable_token_auth и disable_signature в значение false
- Использовать надежные UUID в качестве Bearer токенов
- Регулярно обновлять RSA ключи (не реже чем раз в 6 месяцев)
- Настраивать индивидуальные уровни безопасности для эндпоинтов в соответствии с их критичностью
- Для публичных эндпоинтов использовать уровень "public", для административных - "signature"
- Включить защиту от перебора паролей с разумными ограничениями (5 попыток за 60 минут)
- Хранить конфигурационные файлы с токенами и паролями в защищенном месте с ограниченным доступом

Система безопасности спроектирована для работы в различных сценариях - от строгого режима с полной аутентификацией до упрощенных режимов для разработки, обеспечивая при этом гибкость и надежность защиты данных.

6. Работа с базой данных

Сервер поддерживает подключение к Microsoft SQL Server через драйвер ODBC. Все взаимодействие с базой данных осуществляется асинхронно через хранимые процедуры, что обеспечивает безопасность и производительность системы. Сервер автоматически управляет

соединением с БД, включая установку, проверку работоспособности и корректное закрытие при завершении работы.

6.1 Конфигурирование подключения к базе данных

Настройки подключения к базе данных задаются в конфигурационном файле JSON в разделе "database". Все параметры являются обязательными для успешного подключения, за исключением случаев использования режима запуска без БД.

server	Адрес сервера MSSQL. Может быть доменным именем или IP-адресом. Пример: "sql-server.example.com" или "192.168.1.100"
port	Порт для подключения к SQL Server. По умолчанию используется порт 1433. Должен быть открыт в брандмауэре
database	Имя конкретной базы данных на сервере. Пример: "ZalogDB" или "ClientBase"
username	Имя пользователя для аутентификации в SQL Server. Рекомендуется использовать выделенного пользователя с ограниченными правами
password	Пароль пользователя для доступа к базе данных. Хранится в зашифрованном виде в конфигурационном файле
driver	Название ODBC драйвера. По умолчанию: "ODBC Driver 18 for SQL Server". Должен быть установлен на сервере
connection_timeout	Таймаут подключения в секундах. По умолчанию 10 секунд. Определяет максимальное время ожидания установки соединения

6.2 Дополнительные настройки базы данных

Сервер предоставляет дополнительные параметры для гибкой настройки работы с базой данных, которые влияют на поведение системы при различных сценариях работы.

allow_start_without_db	Разрешает запуск сервера без подключения к БД. По умолчанию false. При значении true сервер запустится даже при недоступности БД, но функциональность, требующая БД, будет недоступна
select_top	Ограничение количества записей в SELECT запросах. По умолчанию 1000 записей. Защищает от перегрузки системы при выборке больших объемов данных

6.3 Управление пулом соединений

Сервер использует механизм пула соединений для оптимизации производительности при частых запросах к базе данных. Настройки пула соединений определяются в секции connection_pooling.

enabled	Включение/отключение механизма пула соединений с базой данных. По умолчанию true - пул включен (рекомендуется для production). false - пул отключен
---------	---

	(каждое соединение создается заново). Рекомендуется включать для высоконагруженных систем.
max_pool_size	Максимальное количество одновременных соединений в пуле. Определяет лимит активных соединений с БД. При превышении лимита новые запросы ожидают освобождения соединения. Значение по умолчанию: 10. Формула расчета: $\text{max_pool_size} = (\text{макс_параллельных_пользователей} \times 0.1) + 5$
min_pool_size	Минимальное количество соединений, поддерживаемых в пуле. Гарантированное количество "горячих" соединений. Ускоряет обработку первых запросов после запуска. Значение по умолчанию: 1. Для нагруженных систем рекомендуемое минимальное значение 2.
connection_lifetime	Время жизни соединения в секундах перед принудительным переподключением. Предотвращает "устаревание" соединений. Помогает при проблемах с сетевыми таймаутами. Значение в секундах (300 = 5 минут)

6.4 Мониторинг состояния базы данных

Сервер включает систему фоновой проверки работоспособности соединения к базе данных. Настройки мониторинга определяются в секции health_check.

enabled	Включение фоновой проверки работоспособности соединения с БД. По умолчанию true - включено.
interval_seconds	Интервал проверки состояния соединения в секундах. Периодичность выполнения тестового запроса "SELECT 1"

6.5 Пример конфигурации подключения к БД

В конфигурационном файле раздел database должен содержать следующую структуру с реальными значениями для вашего окружения:

```
"database": {
  "server": "localhost",           // или IP-адрес вашего SQL Server
  "port": 1433,                   // стандартный порт MSSQL
  "database": "ZalogDatabase",    // имя вашей базы данных
  "username": "zalog_user",       // пользователь БД
  "password": "secure_password",  // пароль пользователя
  "driver": "ODBC Driver 18 for SQL Server", // установленный ODBC драйвер
  "connection_timeout": 10,       // таймаут подключения
  "allow_start_without_db": false, // запрет запуска без БД
  "select_top": 1000              // лимит выборки
}
```


6.6 Проверка подключения к базе данных

Сервер автоматически проверяет подключение к базе данных при запуске и предоставляет несколько методов мониторинга состояния БД:

- При запуске сервер выводит статус подключения к БД в консоль
- Эндпоинт /health/database предоставляет детальную информацию о состоянии подключения
- Сервер периодически проверяет доступность БД через запрос "SELECT 1"
- В логах фиксируются все ошибки подключения и выполнения запросов

6.7 Основные хранимые процедуры

Сервер использует следующие хранимые процедуры для взаимодействия с бизнес-логикой. Все процедуры должны быть созданы в целевой базе данных перед запуском сервера:

USR_Select_ID	Аутентификация пользователя по телефону и паролю. Возвращает данные пользователя в формате JSON
USR_Update	Обновление email и пароля пользователя. Принимает ID пользователя, новый email и хеш пароля
ZbTickets_Json	Получение списка залоговых билетов пользователя. Поддерживает фильтрацию по статусу
Pay_created_at	Обработка платежных операций. Принимает ID транзакции, ID пользователя и JSON с платежами
Doc_created_at	Сохранение метаданных документов. Создает запись о документе с ссылкой на облачное хранилище
Doc_Update_signed	Обновление статуса подписания документа. Изменяет флаг подписания для конкретного документа
DOC_Select_ID	Получение списка документов пользователя. Возвращает метаданные документов в формате JSON
USR_Update_consent_to_mailing	Обновление согласия на email рассылку. Устанавливает флаг согласия для пользователя
USR_Access_Select	Проверка количества неудачных попыток входа. Используется для системы безопасности входа

6.8 Обработка ошибок базы данных

Сервер реализует комплексную систему обработки ошибок при работе с базой данных:

- Таймауты выполнения запросов ограничены 30 секундами для предотвращения блокировок
- Автоматический откат транзакций при ошибках выполнения
- Детальное логирование всех ошибок БД с указанием запроса и параметров
- Грациозная обработка недоступности БД с возвратом понятных сообщений об ошибках
- Валидация параметров запросов для предотвращения SQL-инъекций

При правильной настройке подключения к базе данных сервер обеспечивает надежное и безопасное взаимодействие с данными, поддерживая все необходимые бизнес-процессы системы залоговых операций.

7. Логирование и мониторинг

Сервер реализует многоуровневую систему логирования, которая обеспечивает полный контроль над работой системы и диагностику проблем. Логирование настраивается через конфигурационный файл и поддерживает три независимых канала: базу данных, файловую систему и консоль. Каждый канал может быть настроен индивидуально с указанием уровней логирования и параметров хранения.

7.1 Логирование в базу данных

Система обеспечивает структурированное хранение логов в таблице базы данных для последующего анализа и мониторинга. Логирование в БД включается при наличии подключения к базе данных и соответствующей настройки в конфигурации.

log_to_db	Список уровней логирования для записи в БД. Поддерживаются уровни: INFO, ERROR. Пример: ["INFO", "ERROR"] для логирования всех информационных сообщений и ошибок
mask_sensitive_data	Флаг маскирования конфиденциальных данных. При значении true автоматически скрывает пароли, токены и подписи в логах

Структура таблицы логов в базе данных включает следующие поля:

usr_id	Идентификатор пользователя, связанного с запросом. Может быть NULL для системных запросов
Logs_request_id	Уникальный идентификатор запроса в формате UUID. Позволяет связать запрос и ответ
Logs_endpoint	URL эндпоинта, к которому был выполнен запрос. Пример: "/user/by-phone"
Logs_params	Параметры запроса в строковом представлении. Ограничение длины: 1000 символов
Logs_processing_time_ms	Время обработки запроса в миллисекундах. Позволяет анализировать производительность
Logs_response_code	HTTP статус код ответа. Пример: 200 для успешных запросов, 400 для ошибок
Logs_message	Текст сообщения или ошибки. Ограничение длины: 4000 символов

Логирование в базу данных автоматически маскирует конфиденциальную информацию, такую как пароли, токены авторизации и цифровые подписи. Это обеспечивает безопасность данных при хранении и анализе логов. Запись в БД выполняется асинхронно и не блокирует основной поток обработки запросов.

7.2 Логирование в файл

Файловое логирование обеспечивает надежное хранение логов в файловой системе с автоматическим ротацией по датам. Система самостоятельно управляет созданием файлов и директорий согласно настройкам.

log_to_file	Список уровней логирования для записи в файл. Поддерживаются уровни: INFO, ERROR, WARNING, DEBUG. Пример: ["INFO", "ERROR"] для логирования основных событий и ошибок
log_file_path	Путь к файлу лога. Может быть абсолютным или относительным. Пример: "logs/server.log" создаст файл в директории logs относительно рабочей директории
mask_sensitive_data	Флаг маскирования конфиденциальных данных. Действует глобально для всех каналов логирования

Особенности файлового логирования:

- Автоматическое создание директорий - если указанный путь не существует, система создаст все необходимые директории
- Ротация по датам - каждый день создается новый файл с датой в имени. Пример: server_2024-01-15.log
- Поддержка относительных путей - пути вычисляются относительно текущей рабочей директории сервера
- Кодировка UTF-8 - гарантирует корректное сохранение русскоязычных сообщений
- Автоматическое переключение файлов - при смене даты система автоматически начинает писать в новый файл

Формат записей в файловом логе включает временную метку, уровень логирования и детализированное сообщение. Каждая запись содержит идентификатор запроса, что позволяет отслеживать полный цикл обработки от получения запроса до отправки ответа.

7.3 Консольное логирование

Консольное логирование предоставляет реальный мониторинг работы сервера с улучшенной визуализацией через цветное кодирование и форматированный вывод. Данный тип логирования активируется только в verbose-режиме работы сервера.

verbose_mode	Флаг подробного режима работы. Активируется при запуске с параметром --verbose. Включает детальное логирование в консоль
--------------	--

Возможности консольного логирования:

- Цветовая идентификация статусов - зеленый для успешных операций, красный для ошибок, синий для информационных сообщений
- Детальный вывод HTTP запросов - отображает полный URL, заголовки, параметры и тело запроса

- Детальный вывод HTTP ответов - показывает статус код, заголовки ответа и тело ответа
- Временные метки - каждая операция сопровождается точным временем выполнения
- Визуальные разделители - запросы и ответы отделяются линиями для улучшения читаемости

Консольное логирование особенно полезно при отладке и первоначальной настройке сервера, так как предоставляет мгновенную обратную связь о всех операциях системы. В рабочем режиме рекомендуется отключать verbose-режим для снижения нагрузки и улучшения производительности.

7.4 Уровни логирования и фильтрация

Система поддерживает гибкую настройку уровней логирования для каждого канала независимо. Это позволяет оптимизировать объем хранимых данных и сосредоточиться на важных событиях.

DEBUG	Детальная отладочная информация. Используется для диагностики сложных проблем. Не рекомендуется для рабочих окружений
INFO	Информационные сообщения о нормальной работе системы. Включает данные о запросах, ответах и бизнес-операциях
WARNING	Предупреждения о потенциальных проблемах. Система продолжает работать, но требует внимания администратора
ERROR	Сообщения об ошибках, требующих вмешательства. Включает ошибки базы данных, сетевые проблемы и сбои бизнес-логики

Фильтрация сообщений происходит на уровне каждого канала логирования. Сообщение записывается в канал только если его уровень входит в разрешенный список для данного канала. Например, при настройке ["ERROR"] для файлового логирования, в файл будут записываться только сообщения об ошибках.

7.5 Мониторинг состояния системы

Сервер предоставляет специализированные эндпоинты для мониторинга состояния всех компонентов системы. Эти эндпоинты доступны без аутентификации и предоставляют детальную информацию о работе сервера.

/health	Общая информация о состоянии сервера. Включает статус БД, безопасности, логирования и сетевых настроек
/health/security	Детальная информация о режиме безопасности. Анализирует текущие настройки и предоставляет диагностику
/health/database	Статус подключения к базе данных. Показывает версию сервера, настройки и доступность
/health/logging	Информация о конфигурации логирования. Отображает активные каналы и их настройки
/health/network	Сетевые настройки и конфигурация CORS. Список доступных эндпоинтов

/health/stat	Отображение статистики запросов
--------------	---------------------------------

Система мониторинга обеспечивает круглосуточный контроль над работой сервера и позволяет оперативно реагировать на возникающие проблемы. Все эндпоинты здоровья возвращают данные в структурированном JSON формате, что удобно для интеграции с системами мониторинга и оповещения.

8. API эндпоинты

Сервер предоставляет REST API для взаимодействия с клиентскими приложениями и системами. Все эндпоинты защищены многоуровневой системой аутентификации и авторизации, поддерживают единый формат ответов и гарантированно возвращают серверную подпись в заголовках ответа.

8.1 Эндпоинты мониторинга и диагностики

GET /health

Назначение: Основной эндпоинт проверки жизнеспособности системы. Возвращает сводную информацию о состоянии всех компонентов сервера.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела, не требует заголовков аутентификации

Структура ответа:

status	Общий статус системы: healthy/warning/unhealthy
server	Информация о сервере: version, host, port, uptime, verbose_mode, debug_mode
database	Состояние БД: status, server, name, select_top_limit
security	Настройки безопасности: certificates, token_auth, signature_verification, mode
logging	Логирование: file_logging, database_logging, console_logging, log_level

GET /health/security

Назначение: Детальная диагностика системы безопасности. Анализирует текущий запрос, проверяет корректность заголовков безопасности.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела

Структура ответа:

token_authentication	Настройки токенов: enabled, allowed_tokens_count
signature_verification	Проверка подписей: enabled, signature_ttl_seconds

certificates	Сертификаты: enabled, private_key.status, public_key.status
request_analysis	Анализ запроса: security_diagnostics.recommendations

GET /health/database

Назначение: Мониторинг состояния подключения к базе данных.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела

Структура ответа:

status	Статус подключения: connected/disconnected
connection	Параметры подключения: server, port, database, driver, connection_timeout
settings	Настройки БД: select_top_limit, allow_start_without_db
server_info	Информация о сервере: version, health_check

GET /health/logging

Назначение: Отслеживание состояния системы логирования.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела

Структура ответа:

file_logging	Файловое логирование: enabled, levels, current_log_file, file_path
database_logging	Логирование в БД: enabled, levels
console_logging	Консольное логирование: enabled
verbose_mode	Verbose режим: enabled, description
settings	Настройки: mask_sensitive_data

GET /health/network

Назначение: Сетевая диагностика и информация о конфигурации.

Уровень безопасности: Требуется токен аутентификации (уровень 'token')

Требования к запросу: GET запрос с заголовком Token: Bearer [валидный_токен]

Структура ответа:

server	Серверные настройки: host, port, url
cors	CORS настройки: enabled, allowed_origins, allowed_methods
endpoints	Список доступных эндпоинтов

GET /health/cloud

Назначение: Мониторинг состояния облачного хранилища.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела

Структура ответа:

status	Статус облака: available/unavailable
enabled	Включено ли облачное хранилище
configuration	Конфигурация: url_configured, repo_id_configured, upload_path, timeout, temp_dir
availability_check	Проверка доступности: method, success, description
statistics	Статистика: last_check_time, requests_processed

GET /health/stat

Назначение:

Получение детальной статистики работы сервера, включая метрики производительности, использование ресурсов и аналитику запросов.

Уровень безопасности:

Публичный (не требует аутентификации)

Требования к запросу:

GET запрос без тела

Структура ответа:

Параметр	Тип	Описание
status	string	Общий статус системы: healthy/warning/unhealthy
statistics_period	object	Период сбора статистики: • reset_time - время сброса статистики • uptime_seconds - время работы в секундах • uptime_human - время работы в человекочитаемом формате
database_statistics	object	Статистика базы данных: • total_queries - общее количество запросов

		• max_execution_time_ms - максимальное время выполнения запроса • queries_per_hour - запросов в час • slowest_queries - список самых медленных запросов
cloud_statistics	object	Статистика облачного хранилища: • total_requests - общее количество запросов • successful_requests - успешные запросы • failed_requests - неудачные запросы • success_rate - процент успешных операций
server_statistics	object	Статистика сервера: • total_requests - общее количество запросов • requests_per_hour - запросов в час • hourly_distribution - распределение по часам • daily_statistics - статистика по дням недели • peak_performance - пиковые показатели: ○ peak_hour - пиковый час ○ peak_day - пиковый день
counters_status	object	Состояние счетчиков: • current_total - текущее значение • max_limit - максимальный лимит • usage_percentage - процент использования • status - статус: normal/warning
timestamp	string	Временная метка формирования отчета

Пример успешного ответа:

```
{
  "status": "healthy",
  "statistics_period": {
```



```
"reset_time": "2025-01-15T10:30:00",

"uptime_seconds": 86400,

"uptime_human": "1 day, 0:00:00"

},

"database_statistics": {

  "total_queries": 1500,

  "max_execution_time_ms": 2450,

  "queries_per_hour": 62.5,

  "slowest_queries": [

    {

      "query": "EXECUTE [dbo].[ZbTickets_Json] ...",

      "params": "user_id: 12345",

      "execution_time_ms": 2450,

      "timestamp": "2025-01-15T14:30:22"

    }

  ]

},

"cloud_statistics": {

  "total_requests": 45,

  "successful_requests": 42,

  "failed_requests": 3,

  "success_rate": 93.33

},

"server_statistics": {

  "total_requests": 2500,

  "requests_per_hour": 104.17,

  "hourly_distribution": {

    "2025-01-15 14:00": 150,

    "2025-01-15 13:00": 120
```

```
},
"daily_statistics": {
  "Monday": {
    "count": 1800,
    "max_hourly": 210
  }
},
"peak_performance": {
  "peak_hour": {
    "hour": "2025-01-15 14:00",
    "count": 210,
    "date": "2025-01-15"
  },
  "peak_day": {
    "day": "Monday",
    "count": 1800,
    "date": "2025-01-15"
  }
}
},
"counters_status": {
  "current_total": 2500,
  "max_limit": 2147483647,
  "usage_percentage": 0.0001,
  "status": "normal"
},
"timestamp": "2025-01-15T15:45:30"
}
```

8.2 Эндпоинты управления пользователями

POST /user/by-phone

Назначение: Поиск информации о пользователе по номеру телефона.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полем phone

Формат запроса:

phone	string	Обязательный	Номер телефона в любом формате
-------	--------	--------------	--------------------------------

Структура ответа (успех):

status	"success"
code	1
data	Объект с данными пользователя: id, email, surname, name, patronymic

Структура ответа (ошибка):

status	"error"
code	1
message	"Пользователь не найден"

POST /user/update

Назначение: Обновление данных пользователя в базе данных.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями id, email, password

Формат запроса:

id	integer/string	Обязательный	Идентификатор пользователя
email	string	Обязательный	Новый email (пустая строка если не обновляется)
password	string	Обязательный	Новый пароль (пустая строка если не обновляется)

Структура ответа (успех):

status	"success"
--------	-----------

Структура ответа (ошибка):

status	"error"
message	Описание ошибки

POST /user/login

Назначение: Аутентификация пользователя по телефону и паролю.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями phone, password

Формат запроса:

phone	string	Обязательный	Номер телефона пользователя
password	string	Обязательный	Пароль пользователя

Структура ответа (успех):

status	"success"
code	3
data	Объект с данными пользователя: id, phone, email

Структура ответа (ошибка):

status	"error"
code	1 - неверные данные, 2 - учетная запись заблокирована
message	Описание ошибки

POST /user/mailling

Назначение: Управление согласием на email рассылку пользователя.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями user_id, consent_to_mailing

Формат запроса:

user_id	string	Обязательный	Идентификатор пользователя
consent_to_mailing	boolean	Обязательный	Согласие на рассылку: true/false

Структура ответа (успех):

status	"success"
--------	-----------

Структура ответа (ошибка):

status	"error"
message	Описание ошибки

8.3 Эндпоинты работы с залоговыми операциями

POST /ticket/list

Назначение: Получение списка залоговых билетов пользователя.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями user_id, status (опционально)

Формат запроса:

user_id	string	Обязательный	Идентификатор пользователя
status	string	Опциональный	Статус для фильтрации

Структура ответа (успех):

status	"success"
tickets	Массив объектов залоговых билетов

Структура объекта залогового билета:

external_id	Номер залогового билета
type	Тип билета: online/offline
status	Статус билета
start_date	Дата выдачи займа
end_date	Дата окончания займа
benefits_end_date	Дата окончания льготного срока
last_payment_date	Дата последней оплаты
interest_rate_per_day	Процентная ставка в день
interest_rate_per_year	Процентная ставка в год
amount	Сумма займа
loan_debt	Текущий долг по займу
interest_debt	Текущий долг по процентам
document	Объект документа: id, name, link (только для online)
items	Массив предметов залога: name, status, characteristics

POST /payment/set

Назначение: Обработка данных о платежах.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями transaction_id, user_id, payments

Формат запроса:

transaction_id	string	Обязательный	Уникальный идентификатор транзакции
user_id	string	Обязательный	Идентификатор пользователя
payments	array	Обязательный	Массив объектов платежей
payments[].external_id	string	Обязательный	external_id залогового билета
payments[].date	string	Обязательный	Дата платежа в формате DD.MM.YYYY
payments[].amount	float	Обязательный	Сумма платежа

Структура ответа (успех):

status	"success"
message	"Платежи успешно обработаны"

Структура ответа (ошибка):

status	"error"
message	"Ошибка обработки платежей"

8.4 Эндпоинты работы с документами

POST /document/list

Назначение: Получение списка документов пользователя.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полем user_id

Формат запроса:

user_id	string	Обязательный	Идентификатор пользователя
---------	--------	--------------	----------------------------

Структура ответа (успех):

status	"success"
items	Массив объектов документов

Структура объекта документа:

id	Уникальный идентификатор документа
name	Название документа
date	Дата создания документа (может быть null)
desc	Описание документа (может быть null)
is_signed	Статус подписания: true/false/null

POST /document/load

Назначение: Сохранение документа в облачное хранилище и запись метаданных в базу данных.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями data, extension, name, description, user_id

Формат запроса:

data	string	Обязательный	Данные файла в формате base64
extension	string	Обязательный	Расширение файла (pdf, jpg, png и т.д.)
name	string	Опциональный	Имя файла
description	string	Опциональный	Описание документа
user_id	integer	Опциональный	ID пользователя для привязки документа

Структура ответа (успех):

status	"success"
link	Ссылка на документ в облачном хранилище

POST /document/signed

Назначение: Обновление статуса подписания документа.

Уровень безопасности: Требуется токен и подпись (уровень 'signature')

Требования к запросу:

- Заголовки: Token: Bearer [токен], Signature: [base64-подпись]
- Тело запроса: JSON с полями document_id, is_signed

Формат запроса:

document_id	string	Обязательный	Идентификатор документа в системе
is_signed	boolean	Обязательный	Статус подписания: true - подписан, false - отклонен

Структура ответа (успех):

status	"success"
--------	-----------

Структура ответа (ошибка):

status	"error"
message	Описание ошибки

8.5 Вспомогательные эндпоинты

GET /help

Назначение: Возвращает HTML-страницу с полной документацией по API.

Уровень безопасности: Публичный (не требует аутентификации)

Требования к запросу: GET запрос без тела

Структура ответа: HTML-страница с интерактивной документацией

8.6 Общие требования ко всем запросам

Все защищенные запросы (уровни 'token' и 'signature') должны содержать:

- Заголовок Token: Bearer [валидный_токен]
- Заголовок Signature: [base64-кодированная_подпись] для уровня 'signature'
- Тело запроса в формате JSON для POST запросов

Формат цифровой подписи: token.expiry_timestamp (пример: "abc123.1690000000")

Все ответы сервера содержат заголовки Token и Signature для проверки подлинности ответа.

9. Облачное хранилище

Сервер поддерживает интеграцию с облачным хранилищем для безопасного хранения и управления документами. Облачное хранилище используется для загрузки файлов, создания публичных ссылок и обеспечения надежного доступа к документам через API.

9.1 Назначение и возможности

Основные возможности облачного хранилища:

- Автоматическая загрузка файлов в облачное хранилище при обработке запросов к эндпоинту /document/load
- Создание уникальных публичных ссылок для доступа к загруженным документам
- Проверка доступности облачного хранилища через health-эндпоинт /health/cloud
- Возможность запуска сервера без доступного облачного хранилища для отладки и разработки
- Автоматическое создание временных директорий для промежуточного хранения файлов
- Поддержка таймаутов операций для обеспечения стабильности работы

9.2 Конфигурационные параметры

Настройки облачного хранилища определяются в секции "cloud" конфигурационного файла config.json. Все параметры являются обязательными для корректной работы интеграции.

enabled	boolean	false	Включить интеграцию с облачным хранилищем. При значении true сервер будет пытаться загружать файлы в облако. При значении false все операции с облачным хранилищем пропускаются
url	string	""	Базовый URL облачного хранилища. Должен быть полным URL с протоколом и заканчиваться на слеш. Пример: "https://cloud.example.com/"
username	string	""	Имя пользователя для аутентификации в облачном хранилище. Может быть email адресом или логином в зависимости от требований облачного сервиса
password	string	""	Пароль для аутентификации в облачном хранилище. Рекомендуется использовать сложные пароли и регулярно их обновлять
repo_id	string	""	Идентификатор репозитория в облачном хранилище. Обычно имеет формат UUID. Пример:

			"b46c398f-f9fe-48cf-ae4d-fd1bdb66db84"
upload_path	string	"preview"	Путь для загрузки файлов в облаке. Относительный путь внутри репозитория. Пример: "documents", "uploads", "preview"
timeout	integer	30	Таймаут операций с облаком в секундах. Определяет максимальное время ожидания ответа от облачного сервиса. Рекомендуемое значение: 30-60 секунд
temp_dir	string	"/tmp/cloud_uploads"	Временная директория для хранения файлов перед загрузкой. Должна быть абсолютным путем к директории с правами на запись. Автоматически создается при необходимости
allow_start_without_cloud	boolean	false	Разрешить запуск сервера без доступного облачного хранилища. При значении true сервер запустится даже если облачное хранилище недоступно. При значении false сервер не запустится при недоступном облачном хранилище

9.3 Пример конфигурации

Полный пример конфигурации облачного хранилища для продакшн-среды:

```
"cloud"    {
    "enabled": true,
    "url": "https://cloud.comul.ru/",
    "username": "edo-zb@comul.ru",
    "password": "AnvIEucYlwAKQMVix9",
    "repo_id": "b46c398f-f9fe-48cf-ae4d-fd1bdb66db84",
    "upload_path": "test",
    "timeout": 30,
    "temp_dir": "/home/user/project/uploads",
    "allow_start_without_cloud": false
}
```

9.4 Процесс работы с облачным хранилищем

При получении запроса на загрузку документа через эндпоинт `/document/load` сервер выполняет следующую последовательность действий:

1. Проверяет параметр "enabled" в конфигурации облачного хранилища
2. Декодирует данные файла из формата base64 во временный файл в директории "temp_dir"
3. Выполняет аутентификацию в облачном хранилище используя "username" и "password"
4. Загружает файл в облачное хранилище по пути "upload_path" в репозитории "repo_id"
5. Создает публичную ссылку на загруженный файл используя базовый "url"
6. Сохраняет метаданные документа и ссылку в базе данных
7. Возвращает клиенту ссылку на загруженный документ

9.5 Мониторинг состояния облачного хранилища

Состояние облачного хранилища можно проверить через эндпоинт `GET /health/cloud`. Эндпоинт возвращает следующую информацию:

status	Общий статус облачного хранилища: "available" или "unavailable"
enabled	Включено ли облачное хранилище в конфигурации
configuration	Текущая конфигурация: url_configured, repo_id_configured, upload_path, timeout, temp_dir
availability_check	Результаты проверки доступности: method, success, description
statistics	Статистика операций: last_check_time, requests_processed

9.6 Рекомендации по настройке

Для обеспечения надежной работы облачного хранилища рекомендуется:

- Установить "allow_start_without_cloud": false в продакшн-среде для гарантии доступности хранилища
- Использовать "timeout": 30-60 секунд для баланса между производительностью и стабильностью
- Настроить регулярное обновление паролей доступа к облачному хранилищу
- Обеспечить достаточное дисковое пространство в "temp_dir" для временного хранения файлов
- Регулярно проверять состояние облачного хранилища через эндпоинт `/health/cloud`
- Использовать отдельные репозитории для тестовой и продакшн-сред
- Настроить мониторинг доступности облачного хранилища в системах наблюдения

9.7 Обработка ошибок

Сервер обрабатывает следующие типы ошибок облачного хранилища:

- Ошибки аутентификации - неверные username или password
- Ошибки сети - недоступность облачного сервиса
- Ошибки таймаута - превышение времени ожидания ответа
- Ошибки дискового пространства - недостаток места во временной директории
- Ошибки загрузки файлов - проблемы при передаче данных в облако

При возникновении ошибок сервер логирует детальную информацию для диагностики и возвращает соответствующие HTTP статусы клиентам.

10. Диагностика и устранение неисправностей

10.1 Проверка состояния сервера

Используйте эндпоинт /health для получения полной информации о состоянии системы:

- Статус сервера и время работы
- Доступность базы данных
- Состояние систем безопасности
- Настройки логирования
- Сетевая конфигурация

10.2 Типичные проблемы и решения

Ошибка подключения к БД	Проверьте строку подключения и доступность MSSQL Server
Ошибка загрузки ключей	Убедитесь в правильности путей к RSA ключам
Ошибки аутентификации	Проверьте токены и цифровые подписи
Проблемы с логированием	Проверьте права доступа к файлам и директориям

10.3 Логи для отладки

Включите verbose режим для детальной отладки:

- `python3 server.py config.json --verbose`
- Просматривайте вывод в консоли
- Анализируйте файлы логов
- Проверяйте записи в базе данных